Homework 7 - Charlie Caravati

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
```

1.

$p(x) = c_n + c_{n-1}x + x_{n-3}x^2 + \cdots + c_1x^{n-1}$ for $(x_j, y_j) = (x_j, f(x_j)), j = 1, 2, \ldots, n$

(a)

$Vc = y$

$p(x) = c_n + c_{n-1}x + x_{n-3}x^2 + \cdots + c_1x^{n-1}$ for $(x_j, y_j) = (x_j, f(x_j)), j = 1, 2, \ldots, n$

$c_1 + c_2x_1 + c_3x_1^2 + \cdots + c_nx_1^{n-1} = y_1$

$c_1 + c_2x_2 + c_3x_2^2 + \cdots + c_nx_2^{n-1} = y_2$

$c_1 + c_2x_n + c_3x_n^2 + \cdots + c_nx_n^{n-1} = y_n$

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} \end{bmatrix}$$

(b)

In [25]:
```
def poly_interp(N):
  f = lambda x: 1 / (1 + (10 * x) ** 2)
  a, b = -1, 1
  xeval = np.linspace(a, b, 1001)

  plt.figure()
  plt.plot(xeval, f(xeval), 'k--', label='Exact f(x)')

  for n_val in [N - 1, N, N + 1]:
    if n_val < 2:
      continue
    h = (b - a) / (n_val - 1)
    x = np.array([-1 + (i - 1) * h for i in range(1, n_val + 1)])
    y = f(x)

    V = np.vander(x, n_val, increasing=True)
    c = np.linalg.solve(V, y)
    peval = np.vander(xeval, n_val, increasing=True) @ c

    print(f'N = {n_val:2d} | Max p(x) = {np.max(np.abs(peval)):.2f}')

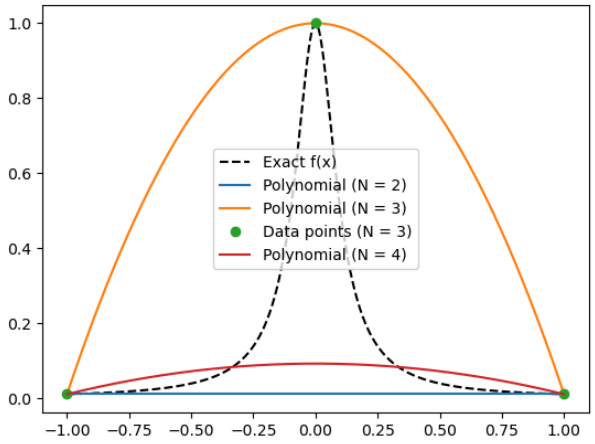
    plt.plot(xeval, peval, label=f'Polynomial (N = {n_val})')

    if n_val == N:
        plt.plot(x, y, 'o', label=f'Data points (N = {n_val})')

  plt.legend()
  plt.show()
```

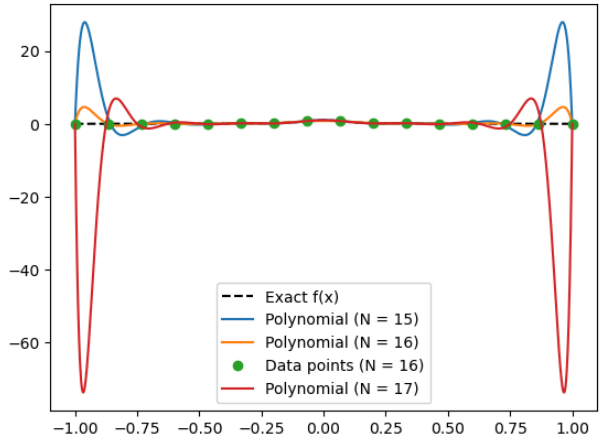
In [31]:
```
poly_interp(3)
```

```
N =  2 | Max p(x) = 0.01
N =  3 | Max p(x) = 1.00
N =  4 | Max p(x) = 0.09
```



In [33]:
```
poly_interp(16)
```

```
N = 15 | Max p(x) = 27.91
N = 16 | Max p(x) = 4.62
N = 17 | Max p(x) = 73.71
```



The interpolation gets progressively worse as N increases. Performance degrades particularly strongly on odd numbered N values, which place a node at the peak x=0 and cause symmetry that drives up error on the boundaries.

2.

$p(x) = \Phi_n(x) \sum_{j=0}^{n} \frac{w_j}{x-x_j} f(x_j)$

or

$p(x) = \frac{\sum_{j=0}^{n} \frac{w_j}{x-x_j} f(x_j)}{\sum_{j=0}^{n} \frac{w_j}{x-x_j}}, x \neq x_j$

where $\Phi_n(x) = \prod_{i=0}^{n}(x - x_j), w_j = \frac{1}{\prod_{i=0, i\neq j}^{n}(x_j - x_i)}$

In [50]:
```
def barycentric(N):
  f = lambda x: 1 / (1 + (10 * x) ** 2)
  a, b = -1, 1

  h = (b - a) / (N - 1)
  x_nodes = np.array([-1 + (i - 1) * h for i in range(1, N + 1)])
  y_nodes = f(x_nodes)

  w = np.zeros(N)
  for j in range(N):
    diffs = x_nodes[j] - x_nodes
    diffs[j] = 1.0
    w[j] = 1.0 / np.prod(diffs)

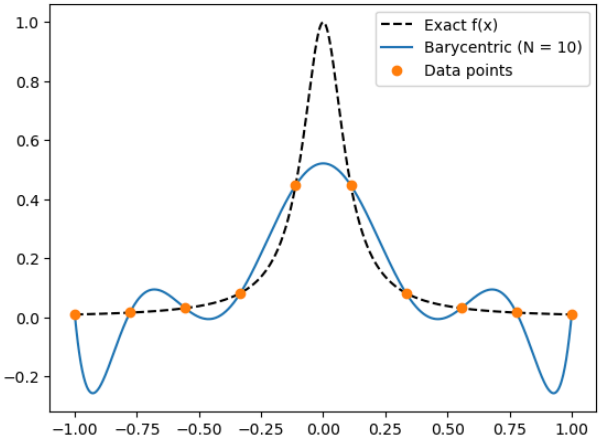
  xeval = np.linspace(a, b, 1001)
  peval = np.zeros_like(xeval)

  for idx, xe in enumerate(xeval):
    match = np.isclose(xe, x_nodes)
    if np.any(match):
      peval[idx] = y_nodes[match][0]
    else:
      terms = w / (xe - x_nodes)
      peval[idx] = np.sum(terms * y_nodes) / np.sum(terms)

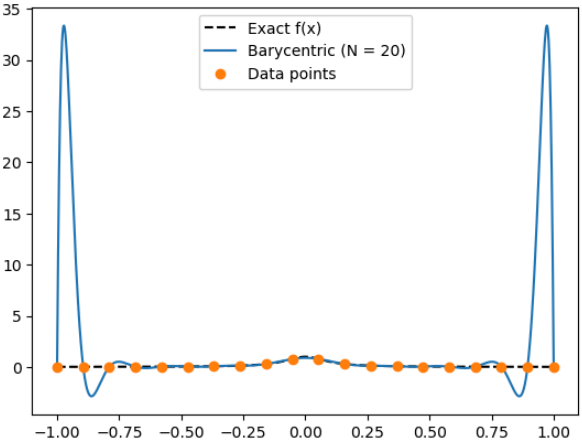
  # Plotting
  plt.figure()
  plt.plot(xeval, f(xeval), 'k--', label='Exact f(x)')
```

```
  plt.plot(xeval, peval, label=f'Barycentric (N = {N})')
  plt.plot(x_nodes, y_nodes, 'o', label='Data points')
  plt.legend()
  plt.show()
```

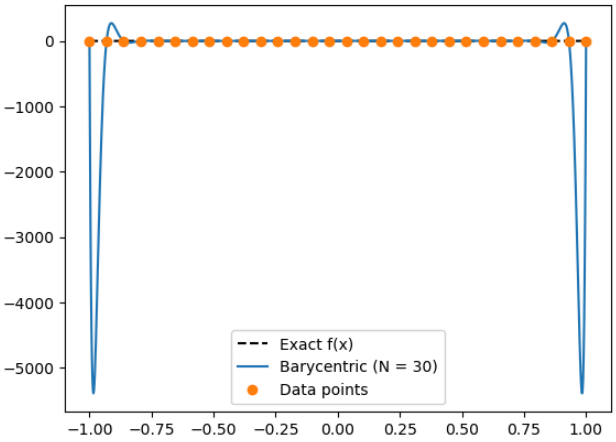
In [53]:
```
barycentric(10)
```



In [51]:
```
barycentric(20)
```



In [52]:
```
barycentric(30)
```



The second barycentric formula still struggles around the boundaries due to Runge's phenomena but works well for small x and large N.

3.

$x_j = cos(\frac{(2j-1)\pi}{2N}), i = 1, \ldots, N$

In [57]:
```
def cheby_bary(N):
  f = lambda x: 1 / (1 + (10 * x) ** 2)
  a, b = -1, 1

  j = np.arange(1, N+1)
  x_nodes = np.cos(((2*j - 1) * np.pi)/(2 * N))
  y_nodes = f(x_nodes)
```

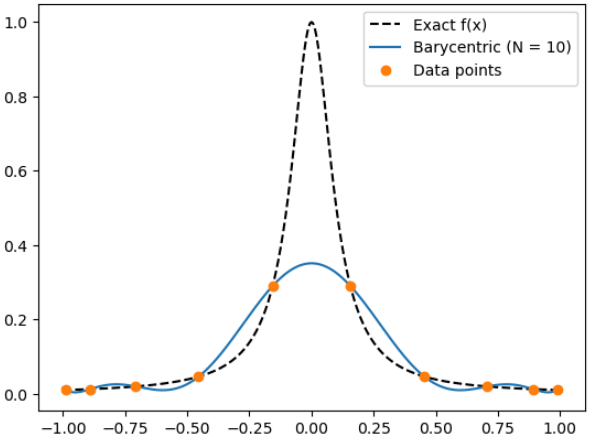
```
w = np.zeros(N)
for j in range(N):
  diffs = x_nodes[j] - x_nodes
  diffs[j] = 1.0
  w[j] = 1.0 / np.prod(diffs)

xeval = np.linspace(a, b, 1001)
peval = np.zeros_like(xeval)

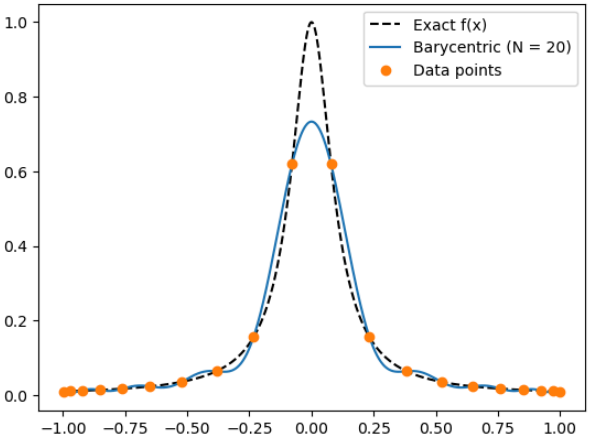
for idx, xe in enumerate(xeval):
  match = np.isclose(xe, x_nodes)
  if np.any(match):
    peval[idx] = y_nodes[match][0]
  else:
    terms = w / (xe - x_nodes)
    peval[idx] = np.sum(terms * y_nodes) / np.sum(terms)

# Plotting
plt.figure()
plt.plot(xeval, f(xeval), 'k--', label='Exact f(x)')
plt.plot(xeval, peval, label=f'Barycentric (N = {N})')
plt.plot(x_nodes, y_nodes, 'o', label='Data points')
plt.legend()
plt.show()
```

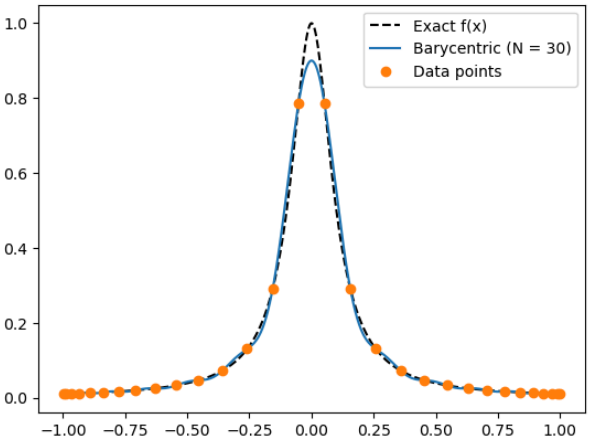
In [59]:
```
cheby_bary(10)
```



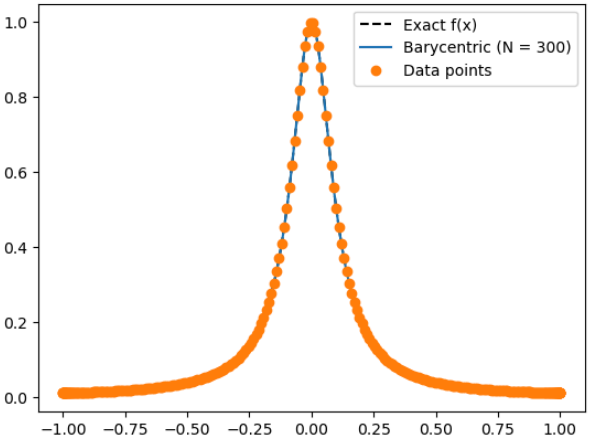
In [60]:
```
cheby_bary(20)
```



In [63]:
```
cheby_bary(30)
```



In [62]:
```
cheby_bary(300)
```



Populating the grid with Chebyshev points, which are clustered around the endpoints, we drastically improve the performance of our interpolation even at very high N values, though the algorithm could suffer from floating point errors in the weight calculations.